

# Continuous-Time Radar-Inertial Odometry with B-Spline Sliding Window Optimization

Timo Weiss

Technical University of Munich (TUM)

timo.weiss@tum.de

**Abstract**—We present a radar-inertial odometry (RIO) system for 6-DOF state estimation on a quadrotor using a single-chip 60 GHz mmWave FMCW radar (10–60 points per frame) and a 1 kHz IMU, evaluated through aggressive racing and attitude-tumbling flight up to  $10 \text{ rad s}^{-1}$  — a regime in which the radar measurement model itself degrades and which existing RIO evaluations do not cover. The state trajectory is represented in continuous time as a quintic position B-spline and a cumulative SO(3) orientation B-spline, enabling asynchronous sensor fusion and closed-form trajectory derivatives. Optimization is performed by a fixed-lag smoother with Schur complement marginalization, running at a 0.3 s stride over a 3 s sliding window. A sensor-only initialization pipeline (gyro integration for orientation, radar dead-reckoning for position) requires no external *pose* reference. Yaw requires a magnetometer (standard on most platforms); here MoCap yaw serves that role.

Radar Doppler measures velocity directly; RIO is therefore evaluated as a *velocity and orientation* provider intended for fusion with position-fixing sensors (GPS, VIO, or SLAM), and position error is reported as drift per distance traveled — the witness of velocity bias. A Markov-blanket factor-selection rule makes the marginalization a true marginal, eliminating prior-weight tuning across flight regimes. A single *dynamics-adaptive measurement weighting law* — rate-adaptive gyroscope trust  $\lambda_g(1+(|\omega|/\omega_0)^4)$ , rate-dependent radar and accelerometer noise inflation, and per-point SNR weighting — covers hover to  $10 \text{ rad s}^{-1}$  tumbling with one configuration, validated on held-out flights never used for tuning. At the *live leading edge* (the estimate a deployed system would emit), velocity and orientation RMSE are  $0.46 \text{ m s}^{-1}/1.9^\circ$  on gentle racing and  $0.32 \text{ m s}^{-1}/2.8^\circ$  on aggressive racing at 0.35–0.70 s per window on a laptop; position drifts 0.6 % and 0.9 % of distance. Backflips — attitude-tumbling flight that exposes radar systematics invisible in level flight — reach  $2.9\%/6.3^\circ$  live. The estimator’s output covariance is NEES-consistent for orientation on racing flights (inflation 0.7–0.9 $\times$ ), supporting its use as a calibrated factor source. We further document negative results, including a diagnostic symptom pattern for marginalization inconsistency, a representation-level analysis that disproves the adaptive-knot-density hypothesis for this regime, and the finding that the apparent radar elevation bias under tumbling is a flight-regime error proxy rather than a physical antenna constant.

**Index Terms**—radar-inertial odometry, continuous-time estimation, B-spline, sliding window, marginalization, mmWave radar, Doppler velocity

## I. INTRODUCTION

Visual odometry and LiDAR-based SLAM are well-established, but both degrade in dusty, foggy, featureless, or textureless environments. Millimeter-wave (mmWave) FMCW radar is inherently robust to these conditions: it penetrates smoke and dust, operates in the dark, and measures Doppler

velocity directly without feature matching. These properties make it attractive for aggressive flight in unstructured environments.

Combining a mmWave radar with an IMU (RIO) for ego-motion estimation has been explored with discrete-time EKF and graph-optimization formulations [1], [2]. However, the asynchronous nature of radar-IMU data (radar at  $\sim 11 \text{ Hz}$ , IMU at  $\sim 1 \text{ kHz}$ ) motivates a continuous-time representation where the trajectory is a smooth function evaluated at any sensor timestamp without interpolation approximation.

Continuous-time estimation with B-splines was introduced for visual-inertial calibration [3] and later applied to various sensor fusion problems [4]. For orientation, the cumulative SO(3) B-spline formulation [5], [6] provides an exact on-manifold representation without re-linearization.

**Research question.** This work asks, end to end: *can a single-chip mmWave radar, fused with an IMU, provide usable 6-DOF state estimation — and if so, what does it take to obtain good results?* The answer is structured in three parts. (1) *What the sensor provides:* per-point Doppler yields direct ego-velocity, but at 10–60 points per frame there are no bearing-stable features for position fixing; yaw is a gauge freedom of Doppler+IMU. The system is therefore a *velocity and orientation* source whose position estimate drifts with distance — by construction, not by defect. (2) *What it takes:* a consistent fixed-lag smoother and a dynamics-adaptive weighting of every measurement stream, each step of which we derive from a measured failure diagnosis rather than tuning folklore. (3) *What the limits are:* characterized via held-out flights, extreme maneuvers, calibrated output covariance, and documented negative results.

We contribute:

- *Single-chip radar under aggressive and tumbling flight*, with a characterization of where and how the radar measurement model breaks in this regime: rate-dependent intra-frame smearing, and the finding that the apparent elevation bias under tumbling is a flight-regime error proxy rather than a physical antenna constant. Existing RIO evaluations cover ground vehicles or gentle UAV flight; backflips at  $10 \text{ rad s}^{-1}$  on a single-chip sensor are, to our knowledge, unexplored.
- A *consistency-analyzed* fixed-lag smoother: a factor-set selection rule that exploits B-spline local support to make the Schur marginal exactly closed — a mechanism distinct from and complementary to FEJ — eliminating

prior-weight tuning; with warm-start alignment and direct marginal evaluation, 0.35–0.70 s per window on a laptop (real time for aggressive racing at a 0.4 s stride).

- One measurement-weighting configuration covering hover to  $10 \text{ rad s}^{-1}$ , with the velocity–orientation trade made explicit as a measured Pareto curve and accuracy validated on *held-out flights* never used for tuning.
- Honest negative results and validation: a diagnostic symptom pattern for marginalization inconsistency; a representation-level study that *disproves* the adaptive-knot-density hypothesis for this regime before implementation; ground-truth quality limits during aggressive maneuvers; and a NEES check showing the reported orientation covariance is consistent on racing flights.

The complete system — physically correct Doppler model with lever arm, sensor-only initialization, full hyperparameter disclosure, deterministic re-runs — is released with code and datasets for independent reproduction.

**Data and code.** The datasets (Vicon MoCap lab, TI IWR6843AOPEVM, Pixhawk) and the C++ Ceres solver with Python bindings used in this work are available at <https://github.com/Dr1zz3l/radar-iwr6843-driver>.

## II. RELATED WORK

**Radar ego-velocity estimation.** Doppler-based ego-velocity estimation from mmWave radar was demonstrated by Kellner et al. [7] and extended to 3D with weighted least squares by Kramer et al. [2], whose RVE system fuses radar with IMU in a discrete-time EKF. Doer et al. [1] present an EKF-based RIO with online extrinsic calibration. Our work uses the same per-frame WLS ego-velocity estimator but embeds it in a continuous-time batch/sliding-window optimizer.

**Continuous-time trajectory estimation.** Furgale et al. [3] introduced uniform B-splines for continuous-time visual-inertial calibration. Anderson and Barfoot [4] generalized this with GP-based “STEAM” trajectories. The basalt VIO [8] uses cumulative B-splines on Lie groups for stereo-inertial odometry. We adapt the basalt-style cumulative SO(3) B-spline to radar-inertial fusion.

**Marginalization in sliding-window estimators.** OKVIS [9] and VINS-Mono [10] pioneered Schur complement marginalization for visual-inertial systems, propagating information from dropped keyframes as a dense Gaussian prior. We apply the same principle to continuous-time B-spline control points and document the practical challenges of prior scaling when information asymmetry spans ten orders of magnitude.

**Closest related work.** Continuous-time radar-inertial odometry was introduced by Ng et al. [11], who fuse Doppler measurements from *multiple automotive* radars with an IMU as a least-squares problem over a B-spline trajectory, evaluated on passenger vehicles. Burnett et al. [12] use a Gaussian-process motion prior for spinning-radar- and lidar-inertial odometry, and Štironja et al. [13] model the IMU trajectory continuously inside an RIO estimator with online spatio-temporal calibration. Single-chip radar-inertial estimation has itself received

recent attention — Huang et al. [14] exploit radar cross-section and Doppler physics to make a sparse single-chip cloud usable, in a discrete-time sliding-window optimization — but on ground-robot and handheld platforms rather than aerial tumbling flight. Our setting differs from all of these in sensor and regime: a *single-chip* radar (10–60 points per frame, 2-TX elevation array) on a quadrotor in aggressive racing and attitude-tumbling flight up to  $10 \text{ rad s}^{-1}$  — a regime in which, as Sec. VII-B shows, the radar measurement model itself degrades in ways invisible on ground vehicles.

**Sliding-window marginalization and consistency.** That naïve sliding-window marginalization is inconsistent is long established in discrete time: Dong-Si and Mourikis [15] analyze which retained states may legitimately connect to marginalized ones, and first-estimates Jacobian (FEJ) methods fix linearization points to preserve the observability nullspace. In continuous time, Ctrl-VIO [16], CLIC [17], and LIO-MARS [18] marginalize spline knots leaving the window, and Hug et al. [6] apply Schur marginalization to visual-inertial splines. Our contribution to this line is narrower and complementary to FEJ: a factor-*set* selection rule that exploits B-spline local support to make the marginal exactly closed (no double counting, no conditioning on interior states), and a documented symptom pattern by which the inconsistent variant can be recognized (Sec. VII-C). In the discrete-time UAV line, Doer and Trommer [1] and extensions to multiple radars [19] and 4D-radar-aided inertial navigation define the state of practice. Rather than comparing numbers across papers — our own quantization ablation (Sec. V) shows firmware configuration alone changes Doppler resolution by  $12\times$  — we cross-validate against this line directly: Sec. VI-F runs our system on the public ICINS-2021 flight datasets of [20] and re-evaluates their EKF estimators under our causal metrics on the same data, and reports the regime-dependent result of porting their filters to our platform.

## III. SYSTEM OVERVIEW

### A. Hardware Platform

The system runs on a quadrotor with a TI IWR6843AOPEVM mmWave radar (60–64 GHz FMCW), a Pixhawk flight controller IMU, and a Vicon MoCap system used for ground truth evaluation. The radar is mounted upside-down with a nominal  $30^\circ$  downward tilt (CAD); the mount is 3D-printed and hand-assembled, so the as-built angle is uncertain by a few degrees and is recovered by online extrinsic calibration (Sec. VI). The nominal extrinsic rotation is  $[180^\circ, 25.5^\circ, 0^\circ]$  (roll, pitch, yaw), translation  $[0.08, 0.02, -0.01] \text{ m}$  in the body frame. The radar operates at  $\sim 11 \text{ Hz}$  with  $0.049 \text{ m/s}$  Doppler resolution (best-velocity firmware configuration). Raw IMU rate is  $\sim 993 \text{ Hz}$ ; both modalities are used at full rate.

### B. Pipeline

Fig. 1 shows the three-stage initialization feeding into a C++ Ceres solver (batch or sliding window): **P1** derives initial roll/pitch from the stationary accelerometer and forward-

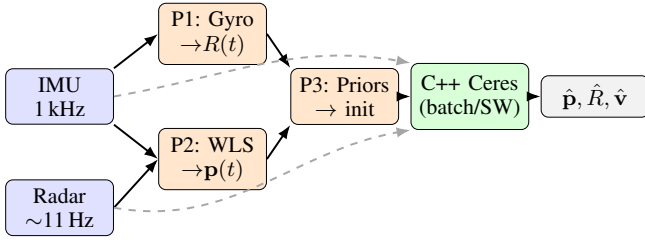


Fig. 1. System pipeline. Solid: initialization flow (P1–P3 seed the optimizer). Dashed: raw sensor data fed as measurement factors. Radar returns first pass a RANSAC ego-velocity prefilter (Sec. IV-B) that rejects elevation-biased outliers before both P2 and the solver. MoCap used only for RMSE evaluation.

integrates gyroscope measurements; **P2** estimates per-frame ego-velocity with WLS and integrates to a position trajectory; **P3** builds sensor-only boundary priors at  $t = 0$ . MoCap data is loaded only for final RMSE evaluation.

#### IV. METHODOLOGY

##### A. State Representation

**Position.** The world-frame position  $\mathbf{p}_w(t) \in \mathbb{R}^3$  is a quintic (order-6) uniform B-spline with knot spacing  $\Delta t_p = 5$  ms. Velocity  $\dot{\mathbf{p}}_w$  and acceleration  $\ddot{\mathbf{p}}_w$  are obtained as analytical derivatives of the spline.

**Orientation.** We use the *cumulative* SO(3) B-spline formulation [5]:

$$\mathbf{R}(t) = \mathbf{R}_{\text{base}}[k-3] \prod_{j=k-3}^k \text{Exp}(\tilde{B}_j(t) \boldsymbol{\Omega}_j), \quad (1)$$

where  $k$  is the active knot index for time  $t$ ,  $\boldsymbol{\Omega}_j \in \mathfrak{so}(3)$  are incremental rotation control points (using the Exp/Log map convention of [21]),  $\tilde{B}_j(t)$  are cumulative cubic basis functions, and  $\mathbf{R}_{\text{base}}$  is the left anchor rotation. Knot spacing is  $\Delta t_o = 8$  ms. Body-frame angular velocity  $\boldsymbol{\omega}_b(t)$  is obtained as a closed-form derivative of (1) directly from `evaluate_lie()` without numerical differentiation.

**Biases and extrinsics.** A constant 6-DoF IMU bias  $[\mathbf{b}_a, \mathbf{b}_g]$  is estimated jointly. The extrinsic pitch  $\delta_{\text{pitch}}$  (1 DoF) is optimized with a soft prior; roll and yaw are fixed (unobservable from Doppler).

##### B. Measurement Models

**Radar Doppler with lever arm.** The TI IWR6843 reports positive Doppler for a receding target. With antenna offset  $\mathbf{t}_{bs}$  from the body CoM, the predicted Doppler for a point at sensor-frame bearing  $\hat{\mathbf{u}}_s$  is:

$$v_D = -\hat{\mathbf{u}}_b^\top (\mathbf{R}_{bw} \mathbf{v}_w(t) + \boldsymbol{\omega}_b(t) \times \mathbf{t}_{bs}), \quad (2)$$

where  $\hat{\mathbf{u}}_b = \mathbf{R}_{bs} \hat{\mathbf{u}}_s$ ,  $\mathbf{R}_{bs}$  is the rotation from radar sensor to body frame (extrinsic [roll=180°, pitch=25.5°, yaw=0°]), and  $\mathbf{R}_{bw} = \mathbf{R}_{wb}^\top$ . The residual  $r_k = v_{D,\text{meas}} - v_{D,\text{pred}}$  uses Huber loss with  $\delta = 1.0$  m/s; the  $0.049 \text{ m s}^{-1}$  Doppler resolution sets a lower bound but does not drive the choice.

**RANSAC ego-velocity prefilter (default front-end).** A single-chip radar with only 2 TX antennas has limited elevation diversity, and a minority of returns carry an elevation-correlated Doppler bias that a robust kernel only down-weights. Following reve [1], we therefore prepend a reve-style 3D least-squares RANSAC ego-velocity estimate to the per-frame Doppler set (inlier threshold 0.15 m/s) and pass only the inlier returns to the spline solver, hard-rejecting the elevation-biased minority before the optimization. The prefilter is seeded (`numpy.default_rng(0)`), so the full pipeline reproduces bit-identically; frames with fewer than five returns bypass it unchanged. It runs once at frame load, not per window, so it does not enter the per-window timing budget (Sec. VI). Sec. VI-F quantifies what it removes — on the public ICINS flights it closes an order-of-magnitude vertical-drift gap, and on our own flights it improves aggressive-racing live position by 22% — and it is enabled by default for every result in this paper.

**Dynamics-adaptive measurement weighting.** The central practical finding of this work is that a single *rate-adaptive trust allocation* across the three measurement streams covers hover to  $10 \text{ rad s}^{-1}$  tumbling with one configuration. Each component was derived from a measured failure diagnosis (Sec. VII-B), not tuned ad hoc:

- *Gyroscope — trust more when rotating fast.* Per-sample weight  $\lambda_g^{\text{eff}} = \lambda_g (1 + (|\mathbf{z}_g|/\omega_g)^p)$  with  $\lambda_g=4$ ,  $\omega_g=4 \text{ rad/s}$ ,  $p=4$ , using the *measured* rate (causal, no spline evaluation). The quartic exponent keeps racing untouched ( $\lambda^{\text{eff}} \approx \lambda_g$  below 3 rad/s) and reaches  $\lambda^{\text{eff}} \approx 160\text{--}500$  during flips, where the gyroscope is the only clean sensor. A quadratic ramp ( $p=2$ ) stiffens the gyro already at 2–5 rad/s — where radar is still informative — and degrades racing roll/yaw.
- *Radar — trust less when rotating fast.* A radar frame integrates over tens of milliseconds; at body rate  $\omega$  the scene smears by  $\omega T_{\text{frame}}$  ( $\approx 17^\circ$  at 10 rad/s). Per-frame noise inflation  $\sigma_{\text{eff}}^2 = \sigma_0^2 (1 + (|\boldsymbol{\omega}|/\omega_r)^2)$ ,  $\omega_r=4 \text{ rad/s}$ ,  $|\boldsymbol{\omega}|$  from the warm-start spline at problem-build time; continuous, no hard-gate tuning cliff.
- *Accelerometer — same form*,  $\omega_a=8 \text{ rad/s}$ : during flips the accelerometer carries thrust transients and vibration, its gravity information is nil mid-flip, and it otherwise distorts orientation through the  $\mathbf{R}\text{--}\ddot{\mathbf{p}}$  coupling.
- *Per-point SNR weighting.*  $w_i = \text{clip}((I_i/\tilde{I})^\alpha, 0.25, 4)$  with frame-median intensity  $\tilde{I}$  and  $\alpha=1$  (median-relative, so the global radar weight is unchanged). High-SNR returns carry measurably better Doppler: on aggressive racing this alone improves live orientation from  $3.31^\circ$  to  $2.88^\circ$  at unchanged position.

The weighting trades are made explicit in Sec. VII-B: on tumbling flight the radar/accelerometer gates exchange velocity accuracy for orientation accuracy along a measured Pareto curve;  $(\omega_r, \omega_a)=(4, 8)$  is its knee.

**Flip-regime Doppler correction.** Under sustained tumbling an additional attitude-correlated Doppler systematic appears,

which we absorb with a per-point term  $v_{\text{corr}} = v_{\text{meas}} - b u_{s,z}$  ( $b = -1.5$  m/s, backflips only). Sec. VII-B shows why  $b$  must be interpreted as a flight-regime error proxy rather than a physical elevation bias of the 2-TX array.

#### Accelerometer.

$$\mathbf{r}_a = \mathbf{z}_a - \mathbf{R}_{bw}(\ddot{\mathbf{p}}_w - \mathbf{g}_w) - \mathbf{b}_a, \quad (3)$$

weighted  $\lambda_a = 0.01$ .

#### Gyroscope.

$$\mathbf{r}_g = \mathbf{z}_g - \boldsymbol{\omega}_b(t) - \mathbf{b}_g, \quad (4)$$

weighted  $\lambda_g = 4.0$  (L2 loss).

#### Gravity direction.

$$\mathbf{r}_{\text{tilt}} = \frac{\mathbf{z}_a - \mathbf{b}_a}{\|\mathbf{z}_a - \mathbf{b}_a\|} - \mathbf{R}_{bw}^\top \hat{\mathbf{g}}, \quad (5)$$

active when  $\|\mathbf{z}_a - \mathbf{b}_a\| - g < 3 \text{ m/s}^2$ . This provides roll/pitch anchoring during near-hover phases without contaminating dynamic flight.

**Heading prior.** Yaw is unobservable from Doppler alone (all yaw-equivalent trajectories predict identical radial velocities under pure rotation about gravity). We supply a heading reference from a magnetometer (standard on most platforms; here simulated by MoCap yaw). The residual  $r_\psi = \psi_{\text{est}} - \psi_{\text{ref}}$  is added at  $\sim 100$  Hz, with  $\lambda_\psi = 0.6$  by default and raised to 10 on the heading-critical bags (slow racing, backflips; Table II footnote).

#### C. Regularization

**Position: minimum snap.**  $E_{\text{snap}} = \lambda_s \int \|\mathbf{p}^{(4)}(t)\|^2 dt$  with  $\lambda_s = 2 \times 10^{-5}$ .

**Orientation: minimum angular acceleration.** Rather than penalizing angular velocity increments (minimum  $\omega$ ), which opposes every banked turn and the entire backflip maneuver, we penalize the *second* finite difference on SO(3):

$$\mathbf{r}_\alpha = \text{Log}(\mathbf{R}_{i-1}^{-1} \mathbf{R}_i) - \text{Log}(\mathbf{R}_i^{-1} \mathbf{R}_{i+1}), \quad (6)$$

which is zero for constant angular rate. Swept  $\lambda_\alpha \in \{0, 0.001, 0.01, 0.1, 1.0\}$ ; 0.1 was best across all bags (1.0 caused backflip divergence).

#### D. Sensor-Only Initialization (P1–P3)

**P1: Orientation.** A stationary segment before flight is detected automatically; the IMU biases  $[\mathbf{b}_a, \mathbf{b}_g]$  are seeded from the gravity residual and mean gyroscope reading during this segment (MoCap orientation is used for a full 3-D accelerometer bias estimate when available; otherwise a gravity-aligned scalar correction is applied). Initial roll/pitch are then derived from the stationary accelerometer (gravity direction). Yaw is set to zero (gauge freedom; corrected later by the heading prior). The full orientation trajectory is obtained by forward-integrating debiased gyroscope readings:  $\mathbf{R}(t + \Delta t) = \mathbf{R}(t) \text{Exp}(\mathbf{z}_g - \mathbf{b}_g) \Delta t$ .

**P2: Position.** Per radar frame, weighted least squares estimates the 3D ego-velocity  $\mathbf{v}_{\text{wls}}$  from all Doppler measurements. Doppler alias unwrapping uses IMU-integrated world-frame velocity to select the correct alias before the main solver

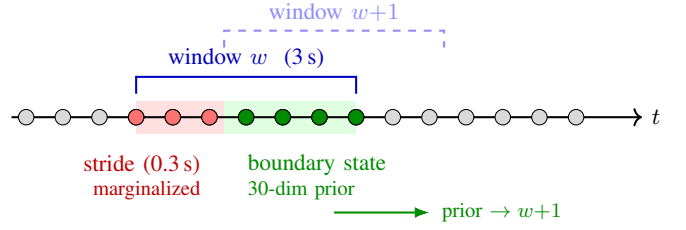


Fig. 2. Sliding window: red CPs (stride zone) are marginalized into a 30-dim Gaussian prior on the green boundary CPs.

runs (20.9% of points aliased in fast racing, 1.7% in slow racing). World-frame velocity is integrated with forward Euler to produce an initial position trajectory.

**P3: Boundary priors.** Position, velocity, orientation, and angular velocity boundary priors are built from the sensor-only state at  $t = 0$ .  $\lambda_{\text{bnd}} = 1000$  for position and velocity.

#### E. Sliding Window with Schur Complement Marginalization

The fixed-lag smoother advances a 3 s window in 0.3 s strides. After each Ceres LM solve, the *stride zone* CPs and orientation knots are marginalized:

$$S = H_{bb} - H_{ab}^\top H_{aa}^{-1} H_{ab}, \quad (7)$$

where  $a$  indexes stride-zone variables and  $b$  indexes boundary variables (last  $N-1$  position CPs, last  $N-1$  orientation knots, and the 6-DoF bias; dimension 30).  $S$  is factored as  $S = LL^\top$  and stored as a `MargPriorFuntor` for the next window.

**Re-centered prior.** Before adding the prior to each window's problem, the linearization point  $\mathbf{x}_0$  is updated to the current warm-start. This makes the prior contribute only curvature (the Hessian shape), not a gradient pull toward a stale historical estimate. The residual is  $\mathbf{r} = L^\top(\mathbf{x} - \mathbf{x}_0)$  in local coordinates.

**Consistent factor selection (Markov-blanket rule).** Which residuals enter Eq. (7) determines whether the prior is a true marginal. A naïve choice — all residuals touching marginalized *or* boundary variables — pulls in every IMU factor of the window through the shared bias block ( $\sim 25,000$  residual rows), *conditions* on interior states (treating them as perfectly known), and double-counts factors that are re-added in the next window. The resulting prior is overconfident by orders of magnitude and must be deflated by a hand-tuned scale of  $10^{-7}$ – $2 \times 10^{-4}$  depending on flight dynamics (Sec. VII-C). We instead exploit B-spline locality: a factor touching marginalized knot  $i$  has support  $[i, i+N-1] \subseteq \text{marg} \cup \text{boundary}$ , so including *only* residuals that touch a marginalized variable (plus the previous prior) yields an exactly closed, consistent marginal. With this rule the prior is applied **unscaled** (scale = 1) across all flight regimes, and its Mahalanobis residual at the solution drops from  $10^6$ – $10^{12}$  to  $\mathcal{O}(1)$  — the prior agrees with the incoming data.

**Relation to FEJ.** This mechanism is distinct from, and complementary to, first-estimates-Jacobian approaches to marginalization consistency [15]: FEJ fixes the *linearization*

TABLE I  
DATASET SUMMARY

| Bag         | Dur. (s) | $v_{\max}$ (m/s) | $ \omega _{\max}$ (rad/s) | Frames |
|-------------|----------|------------------|---------------------------|--------|
| Slow racing | 25.8     | 3.65             | 2.62                      | 258    |
| Fast racing | 18.0     | 5.66             | 9.14                      | 179    |
| Backflips*  | 18.1     | $\sim 4.3$ (95%) | $\sim 10.9$ (95%)         | 181    |

\*95th percentile; MoCap tracking artifacts dominate peaks.

*points* of the prior to preserve the observability nullspace, whereas our rule corrects the *factor set* entering the Schur complement so that the marginal is exactly closed *at a fixed linearization point*. We do not employ FEJ; instead the prior is re-centered at each window’s warm start (curvature retained, gradient pull discarded). We do not claim this re-centering preserves consistency across relinearizations as a theoretical guarantee; rather, in our experiments it left no measurable inconsistency symptom — the NEES analysis of Sec. VI-G provides the corresponding empirical evidence.

**Live-edge warm start.** Control points entering the window each stride would otherwise hold stale dead-reckoned initialization values; any CP-level seam is amplified by the min-snap term ( $\propto \Delta/\Delta t_p^4$ , initial cost  $\sim 10^{13}$ ). We rigidly align the entering segment (shift +  $\Delta R$ ) to the last data-supported CP of the previous solution, reducing the initial cost by five orders of magnitude.

**Marginalization cost.** The restricted Gauss–Newton Hessian for Eq. (7) is accumulated by evaluating the affected cost functions directly (tangent-space Jacobians via the manifold plus-Jacobian, Triggs correction for robust losses), avoiding a full solver re-linearization; the marginalization step costs  $\sim 10$  ms per window.

## V. EXPERIMENTAL SETUP

All experiments use the rosbag datasets collected in a Vicon MoCap lab. Table I summarizes the three bags used for evaluation.

**IMU rate:**  $\sim 993$  Hz (full rate, no downsampling for C++ solver). **Evaluation:** Start-anchored rigid alignment to MoCap ground truth: the alignment rotation  $\mathbf{R}_{\text{align}} = \mathbf{R}_{\text{gt},0} \mathbf{R}_{\text{est},0}^\top$  and translation  $\mathbf{t}_{\text{align}} = \mathbf{p}_{\text{gt},0} - \mathbf{R}_{\text{align}} \mathbf{p}_{\text{est},0}$  are fixed at frame 0 (no scale). The final 3 s is trimmed. Velocity RMSE is computed from the analytical first derivative of the position B-spline, not from finite differences of the estimated trajectory. *Position drift (%)* is position RMSE normalised by the total GT path length — the standard odometry metric for a dead-reckoning system with no loop closure. Start-anchored alignment is harsher than the whole-trajectory SE(3)/Umeyama fit (the EVO/KITTI default) used by most odometry papers, because it cannot retro-fit a constant transform to cancel accumulated drift: it fixes the frame once, at the start, exactly as a deployed estimator must. We adopt it precisely for this causal, deployment-relevant reading, and apply it identically to every system compared. Where we compare against externally published figures (Sec. VI-F) we additionally report whole-

trajectory-aligned numbers so the baselines can be cross-checked against their own papers. *Translational RPE* (relative pose error, Fig. 6) is the KITTI-standard per-segment displacement error averaged over all sub-segments of a given accumulated GT length. Because the start-anchored alignment offsets cancel within every relative displacement, RPE is alignment-independent. For sliding window, *live (leading-edge)* RMSE is computed by snapshotting the estimate at  $[t_{\text{end}} - \text{stride}, t_{\text{end}}]$  of each window at 50 Hz before any subsequent refinement. The *settled* RMSE evaluates the same trajectory after all windows have refined each point.

**Solver implementation.** The Ceres LM problem is solved with SPARSE\_NORMAL\_CHOLESKY (SuiteSparse), up to 400 iterations. All three factor types (gyroscope, accelerometer, and radar Doppler) use hand-derived analytic Jacobians based on the basalt cumulative-spline Jacobian struct; Ceres automatic differentiation was eliminated to avoid Jet-arithmetic overhead over the large set of spline control-point parameters per factor. In the sliding-window solver the linear solve and the marginalization-prior recompute (`compute_prior`) are the dominant costs; Jacobian evaluation accounts for roughly 25–30% of per-window wall time.

## VI. RESULTS

### A. Batch Solver: Offline Ceiling and Extrinsic Self-Calibration

We use the full-trajectory C++ batch solve as an offline accuracy ceiling and as the extrinsic self-calibration stage; the live sliding-window system (Sec. VI-B) is the deployment result and trails this ceiling by the cost of causality. Offline, the batch solver reaches 0.20 m/1.0° on slow racing and 0.73 m/2.5° on fast racing. With the extrinsic pitch left free, it self-calibrates from the 25.5° nominal to 27.0° (slow racing) and 27.2° (fast racing) — recovering the as-built mount angle, which the 3D-printed, hand-assembled fixture leaves a few degrees short of the 30° CAD nominal. Racing bags converge to the same pitch estimate whether initialized at 25.5° or 30°, so the estimate is data-driven, not init-dependent — an independent check on the extrinsic calibration. We lock pitch at 27.5° for the sliding-window runs (Sec. VII-B). The batch estimate is shown for reference (dashed red) in the trajectory overlays (Figs. 3, 4) and the RPE curves (Fig. 6).

### B. Sliding Window: Settled vs. Live

Table II reports sliding window results (3 s window, 0.3 s stride, Schur marginalization). Figures 3 and 4 show trajectory overlays for both bags; Fig. 5 shows the corresponding error timeseries.

Live-edge orientation degrades 0.3–0.4° relative to settled; this is expected because subsequent windows refine previous estimates. With the consistent prior, settled velocity is smooth across stride boundaries (the earlier inconsistent prior produced position jumps that inflated the settled velocity derivative). Per-window solve times are 0.37–0.86 s against a 0.3 s stride on a laptop CPU — fast racing runs in real time at a 0.4 s stride. The dominant cost reductions were: the consistent prior removing the marginalization re-linearization



TABLE II

SLIDING WINDOW: SETTLED AND LIVE LEADING-EDGE RMSE  
(CONSISTENT UNSCALED PRIOR; PER-WINDOW SOLVE TIME ON A LAPTOP CPU)

| Bag                    | Mode             | Vel (m/s)   | Ori (°)     | Pos (m)      | Drift (%)   | $t_{\text{win}}$ (s) |
|------------------------|------------------|-------------|-------------|--------------|-------------|----------------------|
| Slow racing            | Settled          | 0.41        | 1.49        | 0.286        | 0.60        | 0.70                 |
|                        | <b>Live edge</b> | <b>0.46</b> | <b>1.88</b> | <b>0.303</b> | <b>0.63</b> |                      |
| Fast racing            | Settled          | 0.24        | 2.31        | 0.285        | 0.63        | 0.35                 |
|                        | <b>Live edge</b> | <b>0.32</b> | <b>2.84</b> | <b>0.389</b> | <b>0.86</b> |                      |
| Backflips <sup>†</sup> | Settled          | 3.47*       | 4.82        | 1.683        | 3.11        | 0.6                  |
|                        | <b>Live edge</b> | <b>2.35</b> | <b>6.26</b> | <b>1.554</b> | <b>2.87</b> |                      |

All bags use the unscaled consistent marginalization prior (Sec. IV-E) and the *same* dynamics-adaptive weighting law (Sec. IV-B). Per-regime parameters are limited to spline grids ( $\Delta t_p = 5/40/10$  ms), the heading weight, and for backflips the position-init anchor ( $\lambda_{\text{pos-init}} = 0.5$ ) and flip-regime Doppler correction  $b = -1.5$  m/s (Sec. VII-B). Extrinsic pitch locked at  $27.5^\circ$  for all bags. \*Settled velocity on backflips is a retrospective-evaluation artifact (stride-boundary resampling); the live column is the deployment-relevant figure. Drift %: path lengths 48.1 m / 44.9 m / 54.1 m. Iteration-capped configurations reproduce bit-identically across repeated runs; full-convergence configurations vary by  $<2\%$  (multithreaded reduction order).

### slow racing

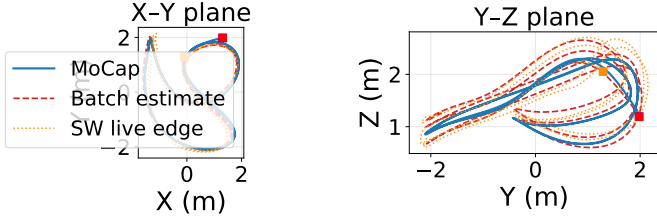


Fig. 3. *slow\_racing*: MoCap ground truth (solid blue), batch estimate (dashed red), SW live edge (dotted orange). X-Y and Y-Z projections; start marked ■.

( $\sim 0.6$  s), coarsening  $\Delta t_p$  from 5 ms to 40 ms for aggressive flight (which *also* drops the LM iteration count from 28 to 11 and slightly improves orientation), and warm-start alignment removing the live-edge seam. For a mapping-oriented profile (full iterations, no warm-start alignment) slow racing reaches 0.314 m / 1.12° settled (yaw  $0.56^\circ$ ) at 2.1 s per window.

### C. Relative Pose Error and Position Drift

Fig. 6 shows the KITTI-style translational RPE as a function of segment length for racing bags (batch and SW live) and backflips (batch only). At longer segments ( $\geq 20$  m) the global B-spline smoother reduces error sharply: slow racing batch reaches 1.12 % at 20 m and 0.43 % at 40 m; fast racing stabilises at  $\approx 3.2\%$  beyond 20 m. At short segments ( $\leq 10$  m) errors are larger (3–7% for racing) because the 11 Hz radar provides only a handful of Doppler measurements per segment and the global optimizer cannot smooth over sub-second intervals. This is a fundamental constraint of the 11 Hz radar rate, not a spline smoothing artifact. Global drift (%) reflects the full-trajectory benefit of the batch optimizer and is the headline deployment metric; RPE captures per-segment local accuracy for a complementary view.

### fast racing

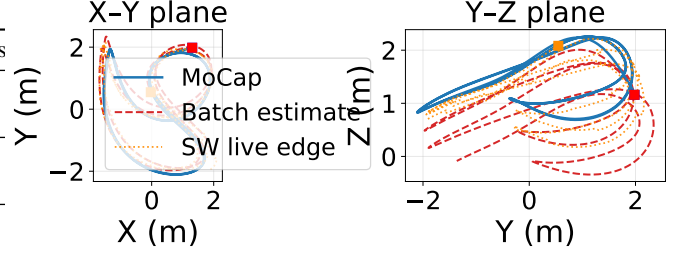


Fig. 4. *fast\_racing*: MoCap ground truth (solid blue), batch estimate (dashed red), SW live edge (dotted orange). X-Y and Y-Z projections; start marked ■. Larger deviations reflect higher dynamics ( $\omega_{\text{max}} = 9.1$  rad/s) compared to slow racing.

TABLE III  
IMU-ONLY VS. FULL RIO (LIVE LEADING-EDGE RMSE)

| Bag         | Sensor     | Vel (m/s)    | Ori (°)     | Pos (m)      | Drift (%)   |
|-------------|------------|--------------|-------------|--------------|-------------|
| Slow racing | IMU only   | 2.014        | 7.55        | 3.061        | 6.36        |
|             | <b>RIO</b> | <b>0.388</b> | <b>2.08</b> | <b>0.337</b> | <b>0.70</b> |
| Fast racing | IMU only   | 1.718        | 6.83        | 6.870        | 15.30       |
|             | <b>RIO</b> | <b>0.487</b> | <b>3.64</b> | <b>0.829</b> | <b>1.85</b> |

Measured with the pre-revision configuration (inconsistent prior, per-bag scale); the isolation of the radar contribution is unaffected. Current live-edge figures are in Table II.

### D. Radar Contribution

Table III isolates the radar contribution by running the same sliding window with `--no-radar`.

Radar reduces live-edge velocity error by 72–81% and position drift by 87–89% (from 6.4% and 15.3% to 0.70% and 1.85% of distance traveled). IMU-only position drifts rapidly through double integration of accelerometer noise; radar Doppler provides direct velocity constraints that bound this drift. Orientation improvement is more modest (47–72%) because the gyroscope already constrains orientation strongly at 1 kHz.

### E. Held-Out Validation

All tuning in this paper used three bags (slow racing, fast racing, backflips). To test generalization, the final configuration was run *unmodified* on flights never used for any decision. On the held-out flights recorded with the same sensor configuration, accuracy matches or beats the tuning bags: a second, independent fast-racing flight reaches 0.37 m / 2.81° live (vs. 0.39 m / 2.84° on the tuning bag), and a circular trajectory reaches 0.88 m / 4.01° live. Five additional stress-tier bags from an earlier recording day use an older radar firmware with  $12\times$  coarser Doppler quantization (0.604 vs. 0.049 m/s), partially unusable accelerometer axes, and per-day time offsets; the default RANSAC prefilter keeps 46–75% of returns even at this quantization (no starvation), and the system degrades gracefully there (1.5–7 m position depending on maneuver), with the *old-firmware backflips*

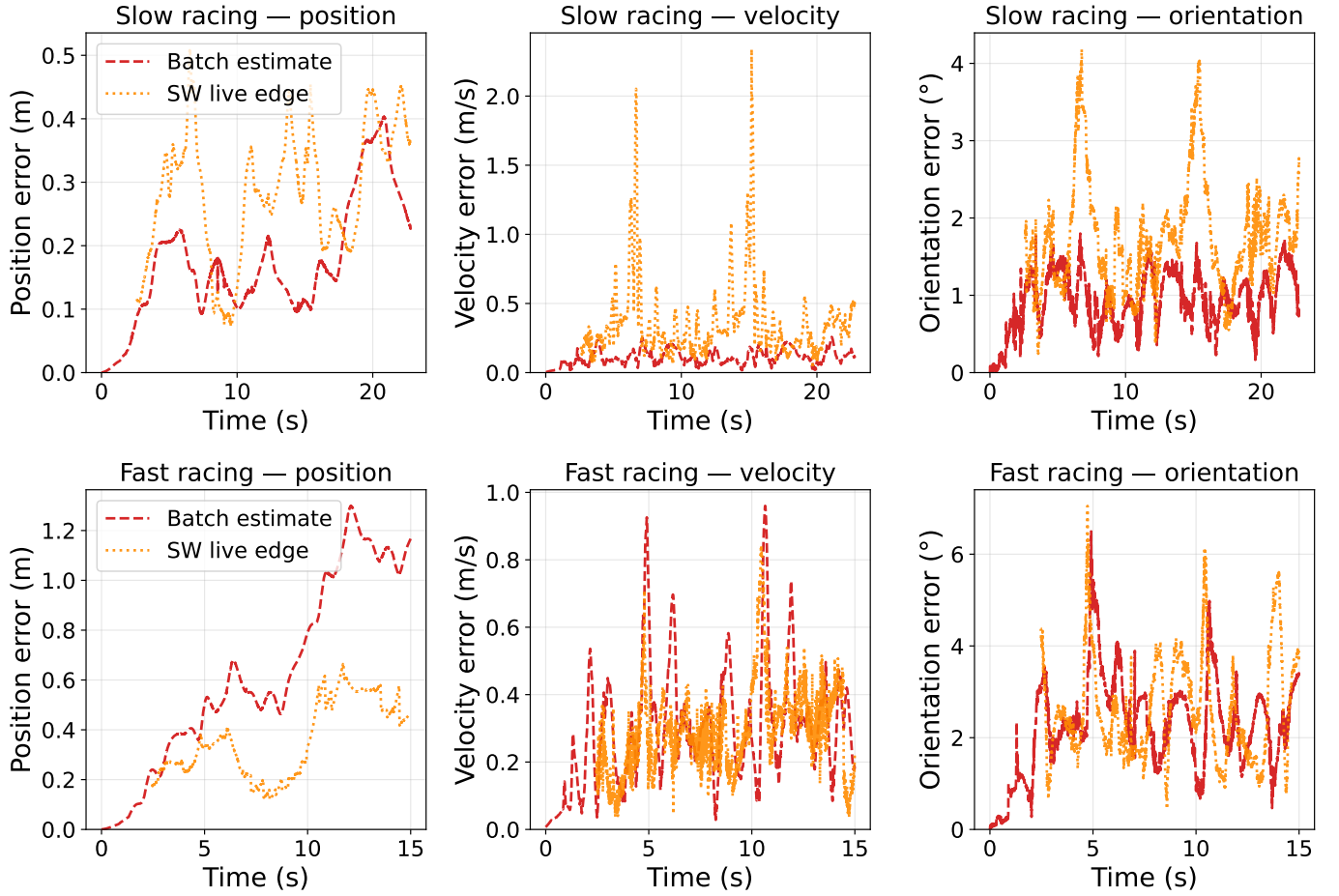


Fig. 5. Position, velocity, and orientation error vs. time for slow racing (top) and fast racing (bottom). Batch estimate (red dashed) and SW live edge (orange dotted). Fast racing position error accumulates monotonically with trajectory length; velocity and orientation errors spike at high-speed, high-dynamics sections.

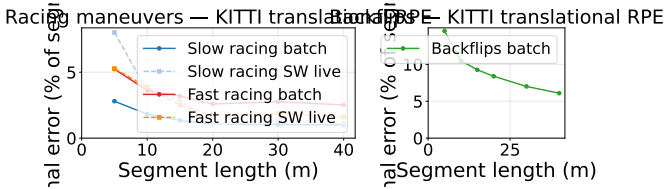


Fig. 6. KITTI-style translational RPE (end-point displacement error as % of segment length) vs. accumulated GT distance. Left: racing bags — batch (solid) and SW live edge (dashed). Right: backflips batch. Errors decrease at longer segments because the global optimizer integrates more sensor data; at short segments the 11 Hz radar provides limited constraints.

orientation reaching  $8.1^\circ$  settled/ $9.1^\circ$  live — better than the  $14.8^\circ$  of its previous, individually tuned best — under the untuned universal configuration. The rate-adaptive weighting thus transfers across both firmware and day. A data-health screening (radar frame-rate continuity, IMU/MoCap coverage) precedes all held-out evaluation, since several early bags contain radar UART dropouts; all evaluation windows lie on healthy spans.

#### F. External Cross-Validation Against EKF Baselines

We cross-validate against the EKF-RIO family of Doer and Trommer [1], [20] in both directions on shared data, with their toolbox pinned at a fixed commit and every parameter deviation documented in the repository.

**Our system on their data.** We run our sliding-window solver, with the universal weighting configuration unmodified, on the four public ICINS-2021 indoor flight datasets [20] (TI IWR6843AOP at 10 Hz, Analog Devices ADIS16448 IMU at 409 Hz with onboard barometer, loop-closure-and-scale-corrected VINS pseudo ground truth), using their published extrinsic calibration (locked) and their  $\pm 60^\circ$  elevation pre-gate. Their estimators (ekf-rio, unaided yaw; ekf-yrio, Manhattan-world radar yaw aiding) are re-run from their stock configurations on the same bags, and *both* systems are evaluated with the identical causal protocol of Sec. V (live estimates, start-anchored alignment, same windows and ground truth). Table IV reports the headline metrics and Table V the decomposition. On position our solver reaches the same order as the baselines (0.5–1.9 m, 0.6–1.3% drift), with velocity RMSE 0.07–0.08 m/s matching theirs. The full 3-DoF orientation column favors our solver (0.51–1.13° vs. up to 10.1°), but

this gap is almost entirely *yaw*: our heading is anchored by a pseudo-magnetometer — here the VINS pseudo-GT yaw used as a magnetometer surrogate, in place of the MoCap yaw available on our own platform — whereas ekf-yrio uses sensor-only Manhattan-world yaw and ekf-r1o is unaided. On the heading-*independent* roll and pitch the three systems are comparable (0.25–0.74°, Table V), so the orientation column is not an attitude win; removing our heading aiding confirms this — on flight\_1 our unaided yaw drifts to 9.9°, matching unaided ekf-r1o (10.1°), while roll/pitch (0.30°) and position are unchanged.

This position parity is what the default RANSAC prefilter (Sec. IV-B) buys, and it is the clearest single demonstration of its effect. A single-chip 2-TX array has limited elevation diversity, and a minority of returns carry an elevation-correlated Doppler bias. Doer’s reve rejects these with 3D RANSAC (inlier 0.15 m/s); a Huber-robust least squares only down-weights them. Measured against the ground-truth attitude, the per-frame vertical ego-velocity bias is −0.06 to −0.16 m/s under a Huber-only front-end but at most 0.10 m/s in magnitude under RANSAC. With the prefilter *disabled*, our vertical channel accumulates a large systematic error (whole-trajectory ATE 9.6 m on flight\_1, 12–15% vertical drift) that the baselines do not exhibit — their vertical drift stays ~0.2–0.3 m; the RANSAC prefilter removes it at the source on all four flights, cutting whole-trajectory ATE to 0.24–0.76 m (Table V) and bringing position to baseline parity. We could not attribute the baselines’ vertical accuracy to altitude aiding: an ablation disabling the barometer in ekf-yrio changes its vertical drift by ≤0.04 m (within run-to-run variation; their stock  $\sigma_{\text{altimeter}}=5$  m makes the baro near-negligible), so the baselines too obtain accurate vertical estimates from radar and IMU alone — by the same outlier-rejection mechanism. The single-chip elevation bias is therefore real but not fundamental: it is an outlier-rejection gap, closed by the same RANSAC stage the baselines use. Under whole-trajectory Umeyama alignment (Table V) all three systems agree to ≤1 m, consistent with the baselines’ published full-alignment figures (they are faithfully reproduced) and confirming that our default front-end matches them.

**Their filters on our data.** The reverse port is regime-dependent, and we report the measured outcome rather than a verdict. Here we port ekf-r1o and its multi-radar generalization x-r1o [22] (run single-radar); ekf-yrio, the yaw-aided single-radar variant Doer benchmarks the ICINS flights with, was the forward-port comparator above. On the slow-racing flight both ekf-r1o and x-r1o fail: their  $\chi^2(3)$  innovation gate rejects 99.7% of the radar ego-velocity updates (301 of 302), so the filter reverts to IMU dead-reckoning and runs essentially unconstrained (> 2 km of error for both, almost entirely vertical,  $\approx \frac{1}{2}gt^2$ ). On the faster flight ekf-r1o instead survives, degrading to 1.4 m (3.1% drift, 9.0°): its larger translational excitation produces radar ego-velocities that clear the gate (only 64% rejected), enough to constrain the filter. The amount of radar information admitted thus scales with how strongly motion excites the sparse Doppler, and the

TABLE IV  
CROSS-VALIDATION ON THE PUBLIC ICINS-2021 FLIGHT DATASETS [20].  
ALL SYSTEMS EVALUATED WITH OUR CAUSAL PROTOCOL (LIVE ESTIMATE, START-ANCHORED ALIGNMENT): POSITION RMSE [M] / VELOCITY RMSE [M/S] / ORIENTATION RMSE [°] / POSITION DRIFT [% OF PATH]. HEADING AIDING DIFFERS BY DESIGN (SEE TEXT); TABLE V DECOMPOSES THESE NUMBERS INTO THEIR HEADING-INDEPENDENT AND PER-AXIS STRUCTURE.

| Flight (path) | ekf-r1o                                | ekf-yrio                                             | Ours                                   |
|---------------|----------------------------------------|------------------------------------------------------|----------------------------------------|
| 1 (142 m)     | 1.41 / 0.20 / 10.1 / 1.0               | <b>0.39</b> / <b>0.08</b> / 0.70 / <b>0.3</b>        | 0.92 / 0.08 / <b>0.51</b> / 0.6        |
| 2 (45 m)      | 0.25 / 0.08 / 1.59 / 0.6               | <b>0.20</b> / <b>0.08</b> / <b>0.91</b> / <b>0.4</b> | 0.51 / 0.08 / 0.94 / 1.1               |
| 3 (147 m)     | <b>0.47</b> / 0.08 / 1.07 / <b>0.3</b> | 0.48 / 0.08 / 1.09 / <b>0.3</b>                      | 1.92 / <b>0.07</b> / <b>0.52</b> / 1.3 |
| 4 (78 m)      | 0.39 / 0.09 / 3.67 / 0.5               | <b>0.22</b> / <b>0.07</b> / 1.41 / <b>0.3</b>        | 0.92 / 0.07 / <b>1.13</b> / 1.2        |

common cause is a stack-level mismatch rather than a defect of their method: our best-velocity firmware yields only ~13 points per frame (their RANSAC/ $\sigma$ -gates expect hundreds, and ego-velocity estimation fails outright on roughly half of all scans), there is no hardware radar trigger, and our racing-frame vibration (measured gyro white-noise PSD 2.1°/s/ $\sqrt{\text{Hz}}$  in flight) exceeds their process-noise parameter range, making propagation overconfident so the gate then rejects the few updates that survive. This is the design-space split: their stack is engineered for dense clouds, hardware triggering, and a low-vibration carrier; ours for sparse, finely quantized Doppler on an aggressive platform.

#### G. Output Covariance Consistency (NEES)

A factor source for a larger fusion graph must report *honest* covariance. We extract the live-edge joint covariance of  $(\mathbf{v}_w, \mathbf{R})$  each window from the converged problem (`ceres::Covariance` over the active support control points, tangent space, marginalization prior included) and compute the normalized estimation error squared (NEES) against MoCap; for a consistent estimator the NEES of each 3-DoF block is  $\chi^2_3$ -distributed with mean 3. On the racing flights the *orientation* NEES mean is 0.7–0.9× its consistent value — the reported covariance is essentially calibrated, a consequence of the measurement weights approximating effective-noise whitening. Velocity NEES medians are 1.4–5.7 (mildly overconfident); their means are dominated by a few windows where the finite-difference MoCap velocity reference itself is unreliable. On backflips the orientation covariance is overconfident by ~3× in standard deviation (unmodeled flip-time error), and only 6 of 50 windows survive the ground-truth quality mask (Sec. V). Deployment recipe: emit the estimator covariance with a per-regime inflation factor (1× gentle, 3× aggressive), or calibrate online from innovation statistics.

#### H. Ablation Studies

Table VI summarizes key design choices.

**Key observations from Table VI.** Full-rate IMU (1 kHz) improves position by 2.9× over 200 Hz, motivating the tight bias priors in `solver_cpp.yaml`. Lever-arm correction helps racing (8–11% position) but has negligible effect on backflips: at  $\Delta t_o=0.8$  ms the dense spline absorbs the  $|\omega \times \mathbf{t}_{bs}| \approx 1$  m/s term through orientation DOF. Schur



TABLE V

DECOMPOSITION OF THE TABLE IV CROSS-VALIDATION, MAKING THE HONEST STRUCTURE VISIBLE AT A GLANCE. *Roll/pitch* IS THE HEADING-INDEPENDENT ATTITUDE RMSE (PER-AXIS RMS, YAW EXCLUDED); THE THREE SYSTEMS ARE COMPARABLE, SO THE LARGE ORIENTATION GAP IN TABLE IV IS YAW, NOT ATTITUDE. *Our drift* SPLITS OUR POSITION ERROR INTO HORIZONTAL AND VERTICAL; WITH THE DEFAULT RANSAC PREFILTER (SEC. IV-B) BOTH ARE SUB-1.3%, THE VERTICAL ELEVATION-BIAS LEAK HAVING BEEN REJECTED AT THE FRONT-END (THE FOOTNOTE GIVES THE PREFILTER-DISABLED FIGURES FOR CONTRAST). *Whole-traj. ATE* RE-EVALUATES EVERY SYSTEM UNDER THE STANDARD EVO/KITTI UMEYAMA ALIGNMENT (NO SCALE) FOR CROSS-CHECKING AGAINST PUBLISHED FIGURES.

| Flight | Roll/pitch RMSE [°] |          |      | Our drift [% path] |       | Whole-traj. ATE [m] |          |      |
|--------|---------------------|----------|------|--------------------|-------|---------------------|----------|------|
|        | ekf-rio             | ekf-yrio | Ours | horiz.             | vert. | ekf-rio             | ekf-yrio | Ours |
| 1      | 0.24                | 0.25     | 0.25 | 0.16               | 0.63  | 0.87                | 0.23     | 0.46 |
| 2      | 0.57                | 0.59     | 0.55 | 0.14               | 1.13  | 0.13                | 0.10     | 0.24 |
| 3      | 0.25                | 0.26     | 0.27 | 0.30               | 1.28  | 0.36                | 0.26     | 0.76 |
| 4      | 0.74                | 0.74     | 0.74 | 0.26               | 1.15  | 0.30                | 0.15     | 0.46 |

Prefilter *disabled* (Huber-only front-end): our vertical drift 12–15%; whole-traj. ATE 9.57/2.88/10.86/5.48 m (flights 1–4) — the elevation-bias leak the default prefilter

marginalization reduces settled position by 48% for fast racing (1.391 m  $\rightarrow$  0.726 m); for slow racing the optimal prior scale is  $10^{-7}$  (near-zero), so both variants give equivalent results — the benefit is captured in Table VII instead. Window duration: 5 s gives best settled position (0.167 m) but slightly worse live edge than 3 s (0.371 vs. 0.338 m); 3 s is the practical optimum for a deployed system (lower latency, better live accuracy). Pitch-only extrinsic optimization is the default; locking pitch to nominal is slightly better for slow racing (0.157 vs. 0.169 m) but the optimizer correctly self-calibrates on fast/backflips bags.

## VII. LIMITATIONS AND NEGATIVE RESULTS

### A. Heading Sensor Requirement

Yaw is a gauge freedom for Doppler: rotating the entire trajectory about the world gravity axis leaves all radial velocity measurements unchanged, so absolute heading cannot be recovered from radar and IMU alone. A magnetometer resolves this and is standard on virtually all flight platforms; here MoCap yaw serves that role. Without it, yaw drifts freely and position direction error accumulates. This is therefore a sensor-architecture note rather than an open research problem.

### B. Extreme Dynamics: Backflips

**Batch.** The batch solver handles backflips only with  $\Delta t_o=0.8$  ms ( $10\times$  denser than racing, 22,630 knots over 18 s) and locked extrinsics (free pitch drifts to  $+18^\circ$  due to dense under-constrained orientation DOF). At this density the batch orientation is *bistable* (detailed below): under small timestamp perturbations it oscillates between  $\sim 8^\circ$  and  $\sim 25^\circ$  regimes, so we do not report a single batch figure and take the sliding-window result as the trustworthy one.

**Sliding window.** The fixed-lag smoother initially appeared fundamentally unsuited to backflips: with the inconsistent prior and a (silently inactive) position anchor, the best result was 2.53 m/10.8° settled. A chain of three fixes revised this verdict. (i) The consistent unscaled prior (Sec. IV-E) restores inter-window information transfer. (ii) A soft position anchor to the dead-reckoned initialization ( $\lambda_{\text{pos-init}}=0.5$  per CP) bounds the position random walk that sparse, rate-inflated radar cannot prevent within a 3 s window (without it position diverges

to 8 m; on racing bags the same anchor is *harmful* — it is a flip-regime rescue, not a general ingredient). (iii) The dynamics-adaptive weighting law (Sec. IV-B) — in particular rate-adaptive gyroscope trust, which on this bag alone is worth  $\sim 3^\circ$  of orientation at zero position cost — plus the flip-regime Doppler correction  $b=-1.5$  m/s. Together these reach **1.68 m/4.8° settled and 1.55 m/6.3° live at  $\sim 0.6$  s per window** (Table II).

*Why  $\Delta t_o=8$  ms in the window.* At the batch density  $\Delta t_o=0.8$  ms a 3 s window has  $\sim 11,250$  orientation DoF against  $\sim 9,000$  gyroscope constraints (1.26 DoF/constraint); the per-window Jacobian is rank-deficient and the solver freezes near its initialization. At 8 ms the window is well-conditioned (62.5 Hz model bandwidth comfortably covers the 10–20 Hz flip dynamics). We further caution that at  $\Delta t_o=0.8$  ms the *batch* problem is bistable: repeated solves under small perturbations (e.g. a few ms of radar-timestamp shift) oscillate between  $\sim 8^\circ$  and  $\sim 25^\circ$  orientation regimes, and the final cost *anti-correlates* with accuracy — the underconstrained problem overfits with the wrong global orientation. Single-run comparisons at this density are not trustworthy.

*Attributing the z-axis failure — and what  $b$  really is.* A visual trajectory check (numeric RMSE conceals it) showed the SW estimate sinking in altitude. Disabling the position anchor exposes the raw behaviour: a near-constant  $-0.57$  m/s world- $z$  drift ( $-8$  m over the flight). Sweeping the per-point correction  $v_{\text{corr}} = v - b u_{s,z}$  produces a linear family of  $z$ -error curves and large negative  $b$  flattens the drift, so we initially interpreted  $b$  as the antenna-fixed elevation bias of the 2-TX array. A joint calibration experiment *disproves* this interpretation: (i) with extrinsic pitch *locked* at its plateau value, level-flight racing degrades catastrophically under any explicit  $b$  (fast-racing position  $0.50 \rightarrow 3.3 \rightarrow 6.7$  m for  $b=0, -0.5, -1.0$ ) — a true antenna-fixed bias would be tolerated, since nothing else can absorb it; and (ii) on backflips the optimum keeps growing monotonically past the WLS-measured static bias of  $-0.5$  m/s (through  $-1.5$  to  $-2.0$ ), into physically implausible territory. We conclude  $b$  is a *flight-regime error proxy* — it absorbs an attitude-correlated Doppler systematic during flips (most likely intra-frame motion distortion) that happens to project like an elevation term — and we cap it at  $b=-1.5$  rather than chase

TABLE VI  
SELECTED ABLATION RESULTS (BATCH SOLVER, SLOW\_RACING UNLESS NOTED)

| Configuration                                                             | Pos (m)              | Ori (°)            |
|---------------------------------------------------------------------------|----------------------|--------------------|
| <i>IMU rate (slow_racing batch, no lever arm<sup>†</sup>)</i>             |                      |                    |
| 200 Hz                                                                    | 0.525                | 1.64               |
| <b>1000 Hz (default)</b>                                                  | <b>0.183</b>         | <b>1.02</b>        |
| <i>Lever-arm correction <math>\omega \times t_{bs}</math> (batch)</i>     |                      |                    |
| No correction — slow racing                                               | 0.183                | 1.02               |
| — fast racing                                                             | 0.862                | 2.73               |
| — backflips                                                               | 1.814                | 8.64               |
| <b>With correction — slow</b>                                             | <b>0.169</b>         | <b>1.02</b>        |
| — fast                                                                    | <b>0.768</b>         | <b>2.64</b>        |
| — backflips                                                               | <b>1.814</b>         | <b>8.44</b>        |
| <i>IMU preintegration vs. raw factors (slow_racing batch)</i>             |                      |                    |
| Forster TRO-2017 preint. at 100 Hz                                        | 0.288                | 6.40               |
| <b>Raw gyro/accel at 1 kHz (default)</b>                                  | <b>0.169</b>         | <b>1.02</b>        |
| <i>Orientation regularization (backflips batch)</i>                       |                      |                    |
| Min- $\omega$ ( $\lambda_\omega=0.001$ )                                  | 1.816                | 8.62               |
| <b>Min-<math>\alpha</math>, <math>\lambda_\alpha=0.1</math> (default)</b> | <b>1.814</b>         | <b>8.44</b>        |
| Min- $\alpha$ , $\lambda_\alpha=1.0$                                      | 2.069                | 83.8               |
| <i>SW window dur. (slow_racing SW, settled / live edge)</i>               |                      |                    |
| 1.5 s                                                                     | 0.488 / 0.675        | 2.14 / 2.47        |
| <b>3.0 s (default)</b>                                                    | <b>0.225 / 0.338</b> | <b>1.57 / 2.08</b> |
| 5.0 s                                                                     | 0.167 / 0.371        | 1.31 / 2.03        |
| <i>Marginalization (fast_racing SW, settled)<sup>‡</sup></i>              |                      |                    |
| None (boundary prior only)                                                | 1.391                | 3.36               |
| <b>Schur complement (default)</b>                                         | <b>0.726</b>         | <b>3.19</b>        |
| <i>Extrinsic optimization (slow_racing batch)<sup>§</sup></i>             |                      |                    |
| <b>Pitch <math>\delta</math> optimized (default)</b>                      | <b>0.169</b>         | <b>1.02</b>        |
| Pitch $\delta$ locked                                                     | 0.157                | 1.02               |

Absolute figures predate the default RANSAC front-end (Sec. IV-B), which shifts the slow\_racing batch default to 0.198 m/1.01° and leaves every ordering here intact. <sup>†</sup>Lever arm disabled for isolation; see Lever-arm rows for the current default (0.169 m/1.02°). <sup>‡</sup>Marginalization compared at fast\_racing where scale =  $2 \times 10^{-4}$  is meaningful; slow\_racing uses scale =  $10^{-7}$  (near-zero), so both variants give equivalent settled results. <sup>§</sup>C++ solver supports only 1-DOF pitch offset (roll/yaw unobservable from Doppler; free 3-angle optimization tested in Python solver showed 6–8° orientation runaway — see Sec. VII-D).

the monotone trend without a mechanism. Relatedly, a  $3 \times 3$  grid over (locked pitch,  $b$ ) shows extrinsic pitch is a *plateau* (26.5–28.5° indistinguishable on backflips), and locking pitch at 27.5° *improves* fast racing over online pitch self-calibration (live position –21%); the earlier “self-calibrated +2° absorbs the elevation bias” reading is retired — 27.5° is simply the correct as-built mount angle (a few degrees short of the 30° CAD nominal; the 3D-printed, loosely-fastened fixture accounts for the difference).

*Remaining gap.* The residual  $\sim 6^\circ$  live orientation error during flips attenuates the reconstructed loop circles relative to ground truth. The radar contribution during flips is fundamentally limited by intra-frame motion smearing; we expect adaptive (non-uniform) knot placement and per-chirp timestamps to be the relevant levers, which we leave to future work.

## backflips

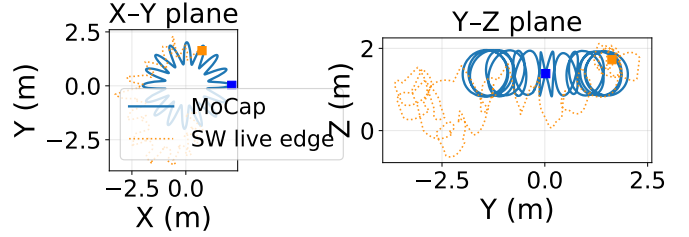


Fig. 7. backflips: MoCap ground truth (solid blue) and SW live edge (dotted orange; consistent prior, soft gate, elevation-bias correction, default RANSAC front-end). The batch estimate is omitted — at  $\Delta t_o=0.8$  ms it is bistable (Sec. VII-B) and the SW live edge is the trustworthy result. X–Y and Y–Z projections; the vertical backflip maneuvers are visible in the Y–Z plane. Start marked  $\blacksquare$ . The estimate follows the overall donut sweep and altitude; the residual  $\sim 6^\circ$  live orientation error through flips attenuates the individual loop circles relative to ground truth.

TABLE VII  
PRIOR SCALE SENSITIVITY (SW, LIVE-EDGE RMSE)

| scale                                      | Slow racing  |             | Fast racing  |             |
|--------------------------------------------|--------------|-------------|--------------|-------------|
|                                            | Pos (m)      | Ori (°)     | Pos (m)      | Ori (°)     |
| $2 \times 10^{-4}$                         | 0.543        | 2.16        | <b>0.825</b> | <b>3.64</b> |
| $10^{-5}$                                  | —            | —           | 0.942        | 3.58        |
| $10^{-6}$                                  | 0.442        | 2.21*       | 0.951        | 3.57        |
| <b><math>10^{-7}</math> (slow default)</b> | <b>0.338</b> | <b>2.08</b> | 1.279        | 3.57        |
| $10^{-8}$                                  | 0.336        | 2.08        | 1.344        | 3.57        |

\*Orientation local maximum:  $2.21^\circ > 2.16^\circ$  ( $2e-4$ ) and  $2.08^\circ$  ( $1e-7$ ). —: scale not evaluated for this bag.

### C. Prior Scale Sensitivity as a Symptom of Inconsistent Marginalization

Our initial marginalization implementation selected all residuals touching marginalized or boundary variables — which, through the shared bias block, baked every IMU factor of the window into the Schur complement, conditioned on interior states, and double-counted factors re-added in the next window (Sec. IV-E). The resulting overconfident prior had to be deflated by a hand-tuned scale, and the required scale varied by orders of magnitude across flight regimes with a harmful intermediate region. We document this behaviour because the symptom pattern — no universal prior weight, non-monotone sensitivity, robust losses on the prior acting backwards — is a useful *diagnostic for marginalization inconsistency* in sliding-window estimators generally. Table VII shows the sweep measured with the inconsistent prior; Fig. 8 plots the orientation sweep.

Slow racing is best served by a near-zero prior ( $10^{-7}$ ); with gentle dynamics, each 3 s window is self-sufficient and additional inter-window coupling is harmful. Fast racing needs the prior ( $2 \times 10^{-4}$ ) for inter-window position continuity — without it (scale =  $10^{-7}$ ) fast racing position degrades from 0.825 m to 1.279 m. For slow racing, scale =  $10^{-6}$  yields a local *orientation* maximum (2.21°; worse than both  $2 \times 10^{-4}$

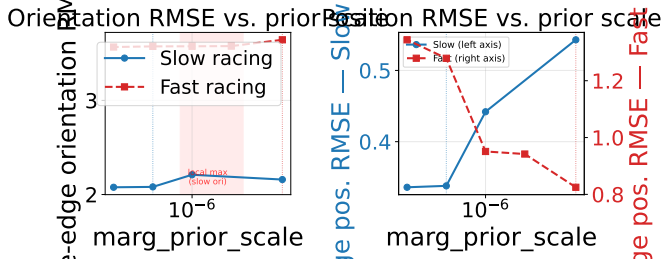


Fig. 8. Live-edge orientation (left) and position (right, dual y-axes) RMSE vs. `marg_prior_scale`. Orientation (left): slow racing shows a local maximum at  $10^{-6}$ ; fast racing orientation is insensitive to scale. Position (right): slow racing worsens monotonically with increasing scale while fast racing improves sharply — the two bags require opposite scale regimes.

at  $2.16^\circ$  and  $10^{-7}$  at  $2.08^\circ$ ), while position improves monotonically with decreasing scale. This partial-constraint effect illustrates why no single scale is universally optimal.

The information asymmetry in the inconsistent  $S$  (orientation eigenvalues  $\sim 5.5 \times 10^{14}$ , position  $\sim 10^4$ ) meant no single scalar could address both modalities; eigenvalue clipping and Cauchy robustification of the prior also failed (the latter counter-productively down-weights the bag that needs the prior most, because aggressive dynamics produce larger prior residuals).

**Resolution.** With the Markov-blanket factor-selection rule of Sec. IV-E, the prior is a true marginal: a single *unscaled* prior (scale = 1) is optimal or near-optimal on all three bags simultaneously, the prior’s Mahalanobis residual at each solution is  $\mathcal{O}(1)$ , and the entire per-mission tuning table becomes obsolete. The fix also improves accuracy: fast-racing settled position improves from 0.727 m (tuned inconsistent prior) to 0.701 m (unscaled consistent prior), and stride-boundary position jumps in the settled trajectory disappear (settled velocity RMSE  $0.89 \rightarrow 0.21$  m/s on slow racing).

#### D. Other Negative Results

**IMU preintegration.** Replacing per-sample gyro/accel factors with Forster TRO-2017 [23] preintegrated factors at 100 Hz reduces constraint density from 8:1 (samples per knot) to 1:1, degrading orientation RMSE from  $1.02^\circ$  to  $6.40^\circ$  and position RMSE from 0.169 m to 0.288 m on slow racing. The smaller Jacobian yields a faster iteration time, but the accuracy loss is unacceptable.

**GNC (Graduated Non-Convexity).** Geman-McClure loss annealing [24] was tested on radar residuals. It improved results only when extrinsics were incorrect (masking calibration errors as outliers); with correct calibration it degraded both bags and added  $3\times$  runtime.

**Full extrinsic optimization.** The C++ solver only supports the 1-DOF pitch offset  $\delta_{\text{pitch}}$  (Sec. IV-A), because Doppler cannot observe roll or yaw. When tested in an earlier Python-based solver that allowed all three angles to optimize freely, roll and yaw drifted  $5\text{--}7^\circ$ , using the unobservable modes as a residual trash can and corrupting orientation by  $6\text{--}8^\circ$ . The C++ design avoids this by construction.

## VIII. CONCLUSION

*Can a single-chip mmWave radar plus IMU provide usable 6-DOF state estimation? Yes* — as a *velocity and orientation* source: at the live leading edge,  $0.46 \text{ m s}^{-1}/1.9^\circ$  on gentle racing,  $0.32 \text{ m s}^{-1}/2.8^\circ$  on aggressive racing (0.35–0.70 s per window on a laptop), confirmed on held-out flights and with NEES-consistent orientation covariance. Absolute position is not observable from Doppler+IMU and drifts with distance (0.6–0.9% on racing, 2.9% under tumbling); yaw requires a heading sensor. Within those structural limits, *good results took three things*: (1) a *consistent* fixed-lag smoother — the Markov-blanket factor-selection rule makes the marginalization a true marginal, eliminating regime-dependent prior tuning whose symptom pattern we document as a reusable diagnostic; (2) a *dynamics-adaptive trust allocation* across the measurement streams (rate-adaptive gyroscope weight, rate-inflated radar/accelerometer noise, per-point SNR weighting) — one configuration from hover to  $10 \text{ rad s}^{-1}$ , with the velocity–orientation trade made explicit as a measured Pareto curve; and (3) *characterized sensor systematics* — including the finding that the apparent elevation bias under tumbling is a flight-regime error proxy, not an antenna constant, and that the extrinsic pitch is a calibration plateau best locked, not estimated online. Notably, the largest accuracy gains came from diagnosis-led re-weighting of existing measurements, not from added model capacity: a representation-level study showed the orientation spline already tracks  $10 \text{ rad s}^{-1}$  flips at the gyroscope noise floor, disproving our own adaptive-knot-density hypothesis before implementation.

Open limitations: (1) loop-scale position geometry through flips is bounded by during-flip radar data quality (Doppler noise  $15\times$  the level-flight core), demonstrated by an asymmetric factor-split ablation; (2) ground truth itself degrades during flips (MoCap occlusion), bounding what can be measured there; and (3) cross-validation against the EKF-RIO family (Sec. VI-F) reveals a design-space split. With our default RANSAC front-end — the same single-chip outlier rejection reveals — our solver reaches position parity on their public flights (0.5–1.9 m, same order as the baselines), matches their roll/pitch, and leads on heading-aided orientation; the prefilter is what closes the single-chip elevation-bias gap, turning a Huber-only port’s 12–15% vertical drift into baseline parity. In the reverse direction their stock filters, starved by our sparse  $\sim 13$ -point clouds and racing-frame vibration, reject up to 99.7% of radar updates and dead-reckon on our slow-racing bag (degrading, not diverging, on the faster one). Each stack is engineered for its own regime — dense clouds and a low-vibration carrier versus sparse, finely quantized Doppler on an aggressive platform.

## REFERENCES

- [1] C. Doer and G. F. Trommer, “An EKF based approach to radar inertial odometry,” in *IEEE International Conference on Multisensor Fusion and Integration (MFI)*, 2020, pp. 152–159.

- [2] A. Kramer, C. Stahoviak, A. Santamaria-Navarro, A.-a. Aghamohammadi, and C. Heckman, "Radar-inertial ego-velocity estimation for visually degraded environments," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 5739–5746.
- [3] P. Furgale, J. Rehder, and R. Siegwart, "Unified temporal and spatial calibration for multi-sensor systems," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2013, pp. 1280–1286.
- [4] S. Anderson and T. D. Barfoot, "Full STEAM ahead: Exactly sparse Gaussian process regression for batch continuous-time trajectory estimation on  $SE(3)$ ," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2015, pp. 157–164.
- [5] C. Sommer, V. Usenko, D. Schubert, N. Demmel, and D. Cremers, "Efficient derivative computation for cumulative B-splines on Lie groups," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [6] D. Hug, P. Bär, R. Siegwart, and A. Cramariuc, "Continuous-time radar-inertial odometry for a 2D FMCW radar," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022.
- [7] D. Kellner, M. Barjenbruch, J. Klappstein, J. Dickmann, and K. Dietmayer, "Instantaneous ego-motion estimation using doppler radar," in *IEEE International Conference on Intelligent Transportation Systems (ITSC)*, 2014, pp. 869–874.
- [8] V. Usenko, N. Demmel, D. Schubert, J. Stückler, and D. Cremers, "Visual-inertial mapping with non-linear factor recovery," *IEEE Robotics and Automation Letters (RA-L)*, vol. 5, no. 2, pp. 422–429, 2020.
- [9] S. Leutenegger, S. Lynen, M. Bosse, R. Siegwart, and P. Furgale, "Keyframe-based visual-inertial odometry using nonlinear optimization," *International Journal of Robotics Research (IJRR)*, vol. 34, no. 3, pp. 314–334, 2015.
- [10] T. Qin, P. Li, and S. Shen, "VINS-Mono: A robust and versatile monocular visual-inertial state estimator," *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.
- [11] Y. Z. Ng, B. Choi, R. Tan, and L. Heng, "Continuous-time radar-inertial odometry for automotive radars," in *IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2021, arXiv:2201.02437.
- [12] K. Burnett, A. P. Schoellig, and T. D. Barfoot, "Continuous-time radar-inertial and lidar-inertial odometry using a gaussian process motion prior," *IEEE Transactions on Robotics*, vol. 41, pp. 1059–1076, 2025, arXiv:2402.06174.
- [13] V.-J. Štironja, L. Petrović, J. Peršić, I. Marković, and I. Petrović, "Radar-inertial odometry with online spatio-temporal calibration via continuous-time IMU modeling," 2026, arXiv:2603.19958.
- [14] Q. Huang, Y. Liang, Z. Qiao, S. Shen, and H. Yin, "Less is more: Physical-enhanced radar-inertial odometry," in *IEEE Int. Conf. Robotics and Automation (ICRA)*, 2024, pp. 15 966–15 972, arXiv:2402.02200.
- [15] T.-C. Dong-Si and A. I. Mourikis, "Consistency analysis for sliding-window visual odometry," in *IEEE Int. Conf. Robotics and Automation (ICRA)*, 2012.
- [16] X. Lang, J. Lv, J. Huang, Y. Ma, Y. Liu, and X. Zuo, "Ctrl-vio: Continuous-time visual-inertial odometry for rolling shutter cameras," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 11 537–11 544, 2022, arXiv:2208.12008.
- [17] J. Lv, X. Lang, J. Xu, M. Wang, Y. Liu, and X. Zuo, "Continuous-time fixed-lag smoothing for lidar-inertial-camera SLAM," *IEEE/ASME Transactions on Mechatronics*, vol. 28, no. 4, pp. 2259–2270, 2023, arXiv:2302.07456.
- [18] J. Quenzel and S. Behnke, "LIO-MARS: Non-uniform continuous-time trajectories for real-time lidar-inertial-odometry," 2025, arXiv:2511.13985.
- [19] J.-T. Huang, R. Xu, A. Hinduja, and M. Kaess, "Multi-radar inertial odometry for 3d state estimation using mmwave imaging radar," in *IEEE Int. Conf. Robotics and Automation (ICRA)*, 2024, arXiv:2311.08608.
- [20] C. Doer and G. F. Trommer, "Yaw aided radar inertial odometry using manhattan world assumptions," in *International Conference on Integrated Navigation Systems (ICINS)*, 2021.
- [21] J. Solà, J. Deray, and D. Atchuthan, "A micro Lie theory for state estimation in robotics," *arXiv preprint arXiv:1812.01537*, 2018.
- [22] C. Doer and G. F. Trommer, "x-RIO: Radar inertial odometry with multiple radar sensors and yaw aiding," *Gyroscopy and Navigation*, vol. 12, no. 4, pp. 329–339, 2021.
- [23] C. Forster, L. Carlone, F. Dellaert, and D. Scaramuzza, "On-manifold preintegration for real-time visual-inertial odometry," *IEEE Transactions on Robotics*, vol. 33, no. 1, pp. 1–21, 2017.
- [24] H. Yang, P. Antonante, V. Tzoumas, and L. Carlone, "Graduated non-convexity for robust spatial perception: From non-minimal solvers to global outlier rejection," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1127–1134, 2020.